# Panduan Penggunaan — HR Intelligence System

Dokumen ini menjelaskan **fitur** dan **cara menjalankan** setiap komponen HR Intelligence System. Ditulis untuk dua pembaca sekaligus:

- **Operator / pengguna** yang ingin menyalakan sistem dan memakainya.
- **Developer** yang ingin tahu komponen mana memanggil komponen mana.

Semua perintah dapat disalin langsung. Jalur yang butuh backend lain ditandai eksplisit di setiap langkah.

---

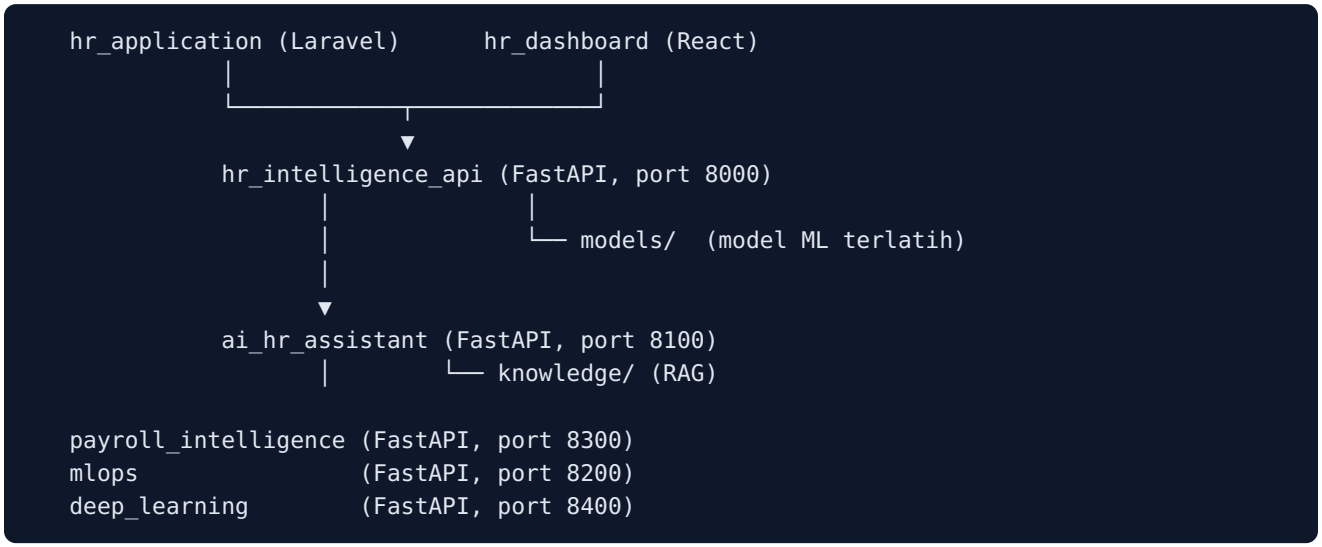
## Daftar Isi


1. Ringkasan sistem
2. Peta komponen dan port
3. Prasyarat
4. Urutan menyalakan sistem
5. HR Intelligence API — mesin prediksi
6. Payroll Intelligence — analitik payroll
7. AI HR Assistant — tanya jawab HR
8. HR Dashboard — visualisasi
9. MLOps — validasi dan monitoring
10. Deep Learning — eksperimen neural network
11. HR Application (Laravel) — proses bisnis HR
12. Contoh alur end-to-end
13. Menjalankan test
14. Troubleshooting
15. Catatan jujur soal batasan


---

## 1. Ringkasan sistem

HR Intelligence System adalah kumpulan modul yang masing-masing bisa berdiri sendiri, tetapi saling terhubung lewat HTTP API.




Poin penting: **hanya `hr_intelligence_api` yang menyimpan model ML.** Modul lain menjadi konsumen, sehingga model cukup dilatih di satu tempat.

---

## 2. Peta komponen dan port

| Komponen | Teknologi | Port | Peran | Butuh backend lain? |
|---|---|---|---|---|
| `hr_intelligence_api` | FastAPI + scikit-learn | **8000** | Prediksi 5 use case ML | tidak |
| `ai_hr_assistant` | FastAPI + RAG | **8100** | Tanya jawab HR + prediksi | ya → 8000 |
| `mlops` | FastAPI | **8200** | Registry, validasi, drift | ya → 8000 |
| `payroll_intelligence` | FastAPI + pandas | **8300** | Analitik & forecast payroll | tidak |
| `deep_learning` | FastAPI + NumPy | **8400** | Eksperimen MLP turnover | tidak |
| `hr_dashboard` | React + Vite | **5173** | Visualisasi | ya → 8000, 8300 |
| `hr_application` | Laravel (PHP) | **8000/ php** | Proses bisnis HR | ya → 8000 |

> **Perhatian port:** `hr_application` (Laravel) secara default juga memakai port 8000. Jika menjalankan keduanya, jalankan Laravel di port lain, misalnya `php artisan serve --port=8080`.

---

## 3. Prasyarat

| Kebutuhan | Versi disarankan | Dipakai oleh |
|---|---|---|
| Python | 3.11+ | semua modul Python |
| Node.js + npm | 18+ | `hr_dashboard` |
| PHP + Composer | 8.2+ / 2.x | `hr_application` |

Cek cepat:

```
python --version
node --version
php --version
```

Install dependensi per modul (masing-masing punya `requirements.txt` sendiri):

```
cd hr_intelligence_api    && pip install -r requirements.txt
cd ../payroll_intelligence && pip install -r requirements.txt
cd ../ai_hr_assistant      && pip install -r requirements.txt
cd ../mlops                && pip install -r requirements.txt
cd ../deep_learning        && pip install -r requirements.txt
```

> `ai_hr_assistant` memerlukan `scikit-learn` dan `numpy` untuk embedding semantik LSA (sudah tercantum di `requirements.txt`).

---

## 4. Urutan menyalakan sistem

Nyalakan dari yang paling dasar. Setiap perintah dijalankan di **terminal terpisah** agar tidak saling memblokir.

```
1. hr_intelligence_api   (port 8000)  ← wajib lebih dulu
2. payroll_intelligence  (port 8300)  ← untuk halaman Payroll dashboard
3. ai_hr_assistant       (port 8100)  ← butuh langkah 1
4. mlops                 (port 8200)  ← butuh langkah 1
5. deep_learning         (port 8400)  ← mandiri
6. hr_dashboard          (port 5173)  ← butuh langkah 1 & 2
7. hr_application        (Laravel)    ← butuh langkah 1
```

Set API key yang sama di semua terminal yang membutuhkannya:

```
export HR_API_KEY=dev-secret
```

---

## 5. HR Intelligence API — mesin prediksi

**Fitur:** melayani prediksi lima use case Machine Learning.

| Use case | Model | Hasil utama |
|---|---|---|
| `turnover` | Soft Voting (ensemble) | `probabilitas_keluar`, `level_risiko` |
| `absensi` | Klasifikasi absen tinggi | `probabilitas_absen_tinggi` |
| `kinerja` | Ordinal (Rendah/Sedang/Tinggi) | `kategori_prediksi`, `skor_ordinal` |
| `payroll` | Deteksi anomali (unsupervised) | `skor_anomali`, `is_anomali` |
| `survival` | Cox Proportional Hazards | `risiko_relatif`, `median_survival_bulan` |

### Menjalankan

```
cd hr_intelligence_api
HR_API_KEY=dev-secret uvicorn app.main:app --port 8000
```

Saat start, log akan menampilkan status pemuatan tiap model. Buka dokumentasi interaktif di http://127.0.0.1:8000/docs.

### Endpoint

| Method | Path | Keterangan |
|---|---|---|
| `GET` | `/health` | Status + daftar model yang termuat |
| `GET` | `/model` | Daftar use case yang tersedia |
| `GET` | `/model/{use_case}/info` | Metadata, metrik, dan versi model |
| `POST` | `/predict` | Prediksi turnover (satu karyawan) |
| `POST` | `/predict/batch` | Turnover (banyak karyawan) |
| `POST` | `/predict/absensi` | Prediksi absen tinggi |
| `POST` | `/predict/absensi/batch` | Batch absensi |
| `POST` | `/predict/kinerja` | Prediksi kategori kinerja |
| `POST` | `/predict/kinerja/batch` | Batch kinerja |
| `POST` | `/predict/payroll` | Deteksi anomali payroll |
| `POST` | `/predict/payroll/batch` | Batch payroll |
| `POST` | `/predict/survival` | Risiko survival-attrition |
| `POST` | `/predict/survival/batch` | Batch survival |

Semua endpoint — termasuk `GET /model*` — memerlukan header `X-API-Key`. Endpoint batch dibatasi `MAX_BATCH_SIZE` karyawan per request.

## Contoh

Setiap use case punya skema input berbeda. Contoh payload di bawah sudah diverifikasi terhadap skema API.

**Turnover** ( `EmployeeInput` ) — field wajib: `employee_id` , `usia` , `masa_kerja_bulan` , `jam_lembur_bulanan` , `gaji_bulanan` , `tanggal_masuk` , `departemen` , `status_kerja` , `pendidikan` .

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "X-API-Key: $HR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "employee_id": "EMP0001",
    "usia": 32,
    "masa_kerja_bulan": 48,
    "jam_lembur_bulanan": 22.0,
    "gaji_bulanan": 9500000,
    "tanggal_masuk": "2021-03-15",
    "departemen": "IT",
    "status_kerja": "Tetap",
    "pendidikan": "S1"
  }'
```

**Absensi** ( `AbsensiInput` ) — field wajib: `employee_id` , `usia` , `masa_kerja_bulan` , `jarak_km` , `beban_kerja` , `lembur_jam_bulanan` , `kepuasan_kerja` , `riwayat_absen_90hari` , `departemen` , `status_kerja` , `pendidikan` .

```
curl -X POST http://127.0.0.1:8000/predict/absensi \
  -H "X-API-Key: $HR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "employee_id": "EMP0001",
    "usia": 32,
    "masa_kerja_bulan": 48,
    "jarak_km": 12.5,
    "beban_kerja": 78.0,
    "lembur_jam_bulanan": 22.0,
    "kepuasan_kerja": 3.2,
    "riwayat_absen_90hari": 4,
    "departemen": "IT",
    "status_kerja": "Tetap",
    "pendidikan": "S1"
  }'
```

**Kinerja** ( `KinerjaInput` ) — field wajib: `employee_id` , `usia` , `masa_kerja_bulan` , `jam_lembur_bulanan` , `jumlah_training_setahun` , `proyek_selesai` , `skor_umpan_balik_360` , `hari_absen_setahun` , `departemen` , `pendidikan` .

```
curl -X POST http://127.0.0.1:8000/predict/kinerja \
  -H "X-API-Key: $HR_API_KEY" -H "Content-Type: application/json" \
  -d '{"employee_id":"EMP0001","usia":32,"masa_kerja_bulan":48,
       "jam_lembur_bulanan":10.0,"jumlah_training_setahun":3,
       "proyek_selesai":7,"skor_umpan_balik_360":4.1,
       "hari_absen_setahun":5,"departemen":"IT","pendidikan":"S1"}'
```

**Payroll** ( `PayrollInput` ) — field wajib: `employee_id`, `periode`, `departemen`, `level`, `gaji_pokok`, `tunjangan`, `lembur_jam`, `tarif_lembur`, `potongan`, `total_gaji`.

```
curl -X POST http://127.0.0.1:8000/predict/payroll \
  -H "X-API-Key: $HR_API_KEY" -H "Content-Type: application/json" \
  -d '{"employee_id":"EMP0001","periode":"2026-01","departemen":"IT",
       "level":"Senior","gaji_pokok":9500000,"tunjangan":1500000,
       "lembur_jam":10,"tarif_lembur":75000,"potongan":500000,
       "total_gaji":11500000}'
```

**Survival** ( `SurvivalInput` ) — field wajib: `employee_id`, `usia`, `gaji_bulanan`, `jam_lembur_bulanan`, `kepuasan_kerja`, `bulan_sejak_promosi`, `jarak_km`, `departemen`, `pendidikan`.

```
curl -X POST http://127.0.0.1:8000/predict/survival \
  -H "X-API-Key: $HR_API_KEY" -H "Content-Type: application/json" \
  -d '{"employee_id":"EMP0001","usia":32,"gaji_bulanan":9500000,
       "jam_lembur_bulanan":22.0,"kepuasan_kerja":3.2,
       "bulan_sejak_promosi":14,"jarak_km":12.5,
       "departemen":"IT","pendidikan":"S1"}'
```

### Bentuk hasil per use case

```
turnover  -> probabilitas_keluar, prediksi_keluar, level_risiko
absensi   -> probabilitas_absen_tinggi, prediksi_absen_tinggi, level_risiko, threshold
kinerja   -> kategori_prediksi, probabilitas{Rendah,Sedang,Tinggi}, skor_ordinal, level
payroll   -> skor_anomali, is_anomali, level, threshold
survival  -> risiko_relatif, level_risiko, median_survival_bulan
```

---

## 6. Payroll Intelligence — analitik payroll

**Fitur:** ringkasan payroll, analitik per departemen/periode, deteksi anomali (IsolationForest), dan proyeksi (forecast) payroll maupun lembur.

### Menjalankan API

```
cd payroll_intelligence
uvicorn src.api:app --port 8300
```

### Endpoint

| Method | Path | Keterangan |
|---|---|---|
| `GET` | `/health` | Status layanan |
| `GET` | `/summary` | Ringkasan payroll |
| `GET` | `/departments` | Agregat per departemen |
| `GET` | `/periods` | Agregat per periode |
| `GET` | `/overtime` | Analitik lembur |
| `GET` | `/components` | Rincian komponen gaji |
| `GET` | `/anomalies` | Daftar anomali + alasannya |
| `GET` | `/forecast/payroll` | Proyeksi payroll |
| `GET` | `/forecast/overtime` | Proyeksi lembur |

### Memakai CLI

```
cd payroll_intelligence

python -m src.cli summary              # ringkasan
python -m src.cli anomalies            # daftar anomali
python -m src.cli forecast --horizon 6 # proyeksi 6 periode
python -m src.cli report               # simpan laporan JSON + Markdown
```

---

## 7. AI HR Assistant — tanya jawab HR

**Fitur:** asisten HR berbasis RAG (Retrieval-Augmented Generation) yang menggabungkan knowledge base kebijakan/prosedur dengan hasil prediksi model ML.

Cara kerjanya:

```
Pertanyaan
   │
   ├─ Router intent ──────────────► pilih jalur (knowledge / prediksi)
   │
   ├─ Retrieval (RAG) ────────────► chunk knowledge base + sitasi
   │     ├─ leksikal (BM25)
   │     ├─ semantik (embedding LSA)
   │     └─ fusion RRF + MMR
   │
   ├─ Tool prediksi (opsional) ──► panggil HR Intelligence API
   │
   ▼
Jawaban + sitasi + saran lanjutan
```

## Menjalankan

```
# Terminal 1 — backend prediksi harus hidup lebih dulu
cd hr_intelligence_api
HR_API_KEY=dev-secret uvicorn app.main:app --port 8000

# Terminal 2 — assistant
cd ai_hr_assistant
cp .env.example .env          # isi HR_INTELLIGENCE_API_KEY=dev-secret
uvicorn src.main:app --port 8100
```

## Endpoint

| Method | Path | Keterangan |
|---|---|---|
| `GET` | `/health` | Status + jumlah dokumen/chunk knowledge |
| `GET` | `/tools` | Daftar tool prediksi yang tersedia |
| `GET` | `/knowledge` | Daftar dokumen knowledge base |
| `GET` | `/models` | Daftar model ML yang bisa dipanggil |
| `POST` | `/chat` | Tanya jawab |

## Contoh

```
# Pertanyaan kebijakan (jalur knowledge)
curl -X POST http://127.0.0.1:8100/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"Bagaimana prosedur pengajuan cuti tahunan?"}'

# Pertanyaan prediksi (jalur tool + model)
curl -X POST http://127.0.0.1:8100/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"Berapa risiko turnover karyawan EMP0001?","employee_id":"EMP0001"}'
```

## Konfigurasi penting (`.env`)

| Variabel | Default | Keterangan |
|---|---|---|
| `LLM_PROVIDER` | `mock` | `mock` (tanpa jaringan) atau `openai` |
| `OPENAI_API_KEY` | kosong | Wajib bila `LLM_PROVIDER=openai` |
| `EMBEDDING_BACKEND` | `lsa` | `lsa` (semantik), `hashing` (leksikal), `neural` |
| `RAG_FUSION` | `rrf` | `rrf`, `linear`, `lexical`, `semantic` |
| `RAG_MMR` | `true` | Buang hasil redundan |
| `RAG_TOP_K` | `4` | Jumlah chunk yang diambil |

> **Penting:** dengan `LLM_PROVIDER=mock` (default), jawaban disusun deterministik dari chunk + hasil tool, **tanpa** memanggil LLM eksternal. Untuk jawaban yang dihasilkan LLM, set `LLM_PROVIDER=openai` dan isi `OPENAI_API_KEY`.

### Menambah pengetahuan

Tambahkan file `.md` ke folder yang sesuai, lalu muat ulang:

```
knowledge/
├── policies/      # kebijakan (cuti, lembur, payroll)
├── procedures/    # prosedur (pengajuan cuti, absensi, reimbursement)
├── regulations/   # regulasi (perlindungan data, kepatuhan AI)
└── faq/           # tanya jawab umum
```

---

## 8. HR Dashboard — visualisasi

**Fitur:** enam halaman analitik.

| Halaman | Isi |
|---|---|
| Ringkasan | KPI utama HR |
| Workforce | Komposisi & pertumbuhan karyawan |
| Turnover | Tingkat turnover & distribusi risiko |
| Attendance | Kehadiran, absen, lembur |
| Payroll | Analitik payroll & anomali |
| HR KPI & Model | Kinerja model ML |

### Menjalankan

```
cd hr_dashboard
npm install
npm run dev          # http://127.0.0.1:5173
```

### Mode data

Dashboard punya dua mode:

- `demo` (default) — memakai data contoh lokal di `src/services/fixtures.js`, sehingga halaman **tidak pernah kosong** walau backend belum menyala.
- `live` — memakai API sungguhan.

Untuk mode live:

```
cp .env.example .env
# ubah VITE_DATA_MODE=live
npm run dev
```

Proxy dev sudah diatur di `vite.config.js`:

```
/api/*          → http://127.0.0.1:8000   (HR Intelligence API)
/payroll-api/*  → http://127.0.0.1:8300   (Payroll Intelligence API)
```

Prefix `/payroll-api` dipakai agar tidak bentrok dengan route SPA `/payroll`.

### Build produksi

```
npm run build
npm run preview
```

---

## 9. MLOps — validasi dan monitoring

**Fitur:** registry model, validasi kelayakan deploy, monitoring metrik, dan deteksi drift.

### Menjalankan

```
cd mlops
uvicorn src.api:app --port 8200
```

### Endpoint

| Method | Path | Keterangan |
|---|---|---|
| `GET` | `/health` | Status layanan |
| `GET` | `/registry` | Seluruh model terdaftar |
| `GET` | `/registry/{use_case}` | Detail satu model |
| `GET` | `/validation` | Hasil validasi semua model |
| `GET` | `/metrics` | Metrik monitoring |
| `GET` | `/metrics/prometheus` | Format Prometheus |
| `POST` | `/monitoring/check-api` | Cek kesehatan HR Intelligence API |
| `GET` | `/drift` | Deteksi feature drift |

### Validasi via CLI

```
cd mlops
python -m src.cli
```

Exit code `1` bila ada model yang gagal validasi — dipakai sebagai gate di CI sebelum model dianggap layak deploy.

### Docker

```
cd mlops
docker compose -f docker/docker-compose.yml up --build
```

Menyalakan HR Intelligence API (8000) dan MLOps API (8200) sekaligus.

---

## 10. Deep Learning — eksperimen neural network

**Fitur:** MLP (Multi-Layer Perceptron) murni NumPy untuk prediksi turnover, dibandingkan dengan baseline klasik. Tanpa PyTorch/TensorFlow.

### Menjalankan API

```
cd deep_learning
uvicorn src.api:app --port 8400
```

| Method | Path | Keterangan |
|---|---|---|
| `GET` | `/health` | Status |
| `GET` | `/metrics` | Metrik MLP |
| `GET` | `/baseline` | Metrik baseline klasik |
| `GET` | `/experiments` | Riwayat eksperimen |

### Memakai CLI

```
cd deep_learning

python -m src.cli train       # latih MLP + catat eksperimen
python -m src.cli baseline    # jalankan baseline klasik
python -m src.cli compare     # bandingkan MLP vs baseline
python -m src.cli experiments # tampilkan run terbaik
```

### Hasil pembanding (dataset turnover saat ini)

```
Model                   ROC AUC        F1
MLP (deep learning)      0.6868    0.5799
Logistic Regression      0.7219    0.6203
Majority class           0.5000    0.0000
```

Pada dataset tabular berukuran kecil ini, **baseline klasik masih unggul**. Modul ini menyediakan titik acuan agar klaim "Deep Learning lebih baik" dapat diuji, bukan diasumsikan.

---

## 11. HR Application (Laravel) — proses bisnis HR

**Fitur:** aplikasi HR utama (data karyawan, absensi, payroll) yang mengonsumsi HR Intelligence API untuk prediksi.

### Konfigurasi

```
HR_INTELLIGENCE_BASE_URL=http://localhost:8000
HR_INTELLIGENCE_API_KEY=dev-secret
```

Endpoint tiap use case diatur di `config/hr_intelligence.php`.

### Memakai facade

```
use App\Facades\HrIntelligence;

HrIntelligence::predictUseCase('absensi', $employeeData);
HrIntelligence::predictUseCaseBatch('payroll', $employeeList);
HrIntelligence::useCaseInfo('survival');
```

### Menjalankan prediksi batch

```
cd hr_application
php artisan hr-intelligence:predict absensi
php artisan hr-intelligence:predict payroll --sync
```

Hasil prediksi disimpan di tabel `hr_intelligence_predictions` dan dapat diakses lewat route admin `admin/hr-intelligence/{useCase}/predictions`.

### Menjalankan aplikasi

```
cd hr_application
composer install
npm install && npm run build
php artisan serve --port=8080    # hindari bentrok dengan HR Intelligence API
```

---

## 12. Contoh alur end-to-end

Skenario: HR ingin tahu risiko turnover seorang karyawan, lalu membaca kebijakan tindak lanjutnya.

```
Terminal 1: hr_intelligence_api  → port 8000
Terminal 2: ai_hr_assistant      → port 8100
Terminal 3: hr_dashboard         → port 5173
```

**Langkah 1 — prediksi langsung ke API**

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "X-API-Key: dev-secret" -H "Content-Type: application/json" \
  -d '{"employee_id":"EMP0001","usia":32,"masa_kerja_bulan":48,
       "jam_lembur_bulanan":22.0,"gaji_bulanan":9500000,
       "tanggal_masuk":"2021-03-15","departemen":"IT",
       "status_kerja":"Tetap","pendidikan":"S1"}'
```

**Langkah 2 — tanya assistant (menggabungkan prediksi + kebijakan)**

```
curl -X POST http://127.0.0.1:8100/chat \
  -H "Content-Type: application/json" \
  -d '{"message":"Berapa risiko turnover karyawan EMP0001 dan apa tindak lanjutnya?",
       "employee_id":"EMP0001"}'
```

Jawaban berisi hasil prediksi model **dan** sitasi dokumen kebijakan.

**Langkah 3 — lihat visualisasi**

Buka http://127.0.0.1:5173, pilih halaman **Turnover** untuk distribusi risiko, lalu **HR KPI & Model** untuk metrik model.

---

## 13. Menjalankan test

Setiap modul punya test sendiri:

```
cd hr_intelligence_api   && python -m pytest tests/ -q   # 32 passed
cd ../ai_hr_assistant    && python -m pytest tests/ -q   # 43 passed
cd ../deep_learning      && python -m pytest tests/ -q   # 17 passed
cd ../payroll_intelligence && python -m pytest tests/ -q # 11 passed
cd ../mlops              && python -m pytest tests/ -q   # 11 passed
```

Test `ai_hr_assistant` memakai `MockLLM` dan `httpx.MockTransport`, sehingga tidak membutuhkan jaringan maupun backend yang hidup.

---

## 14. Troubleshooting

| Gejala | Penyebab umum | Solusi |
|---|---|---|
| `Connection refused` ke port 8000 | HR Intelligence API belum jalan | Nyalakan lebih dulu (bagian 5) |
| `401` / "API key tidak valid" | `X-API-Key` tidak cocok | Samakan `HR_API_KEY` dan `HR_INTELLIGENCE_API_KEY` |
| Model `tidak siap` di log | File model tidak ditemukan | Pastikan folder `models/` ada dan lengkap |
| Dashboard halaman kosong | Backend mati & mode `live` | Pakai mode `demo`, atau nyalakan backend |
| Dashboard `ECONNREFUSED 8300` | Payroll API belum jalan | Nyalakan `payroll_intelligence` (bagian 6) |
| Assistant menjawab template | `LLM_PROVIDER=mock` | Set `LLM_PROVIDER=openai` + `OPENAI_API_KEY` |
| `empty vocabulary` saat start assistant | Knowledge base kosong | Pastikan folder `knowledge/` berisi `.md` |
| Port bentrok Laravel vs API | Keduanya memakai 8000 | Jalankan Laravel di `--port=8080` |
| `sentence-transformers` tidak ada | `EMBEDDING_BACKEND=neural` | Pakai `lsa` (default) atau install paketnya |

## 15. Catatan jujur soal batasan

Bagian ini penting agar ekspektasi sesuai dengan implementasi nyata.

1. **Embedding RAG default adalah `lsa` (TF-IDF + SVD), bukan neural.** Ia jauh lebih baik daripada pencocokan leksikal, tetapi tetap berbasis statistik korpus. Untuk kualitas semantik terbaik termasuk lintas bahasa, aktifkan `EMBEDDING_BACKEND=neural` (butuh `sentence-transformers`).
2. **LLM default adalah `mock`.** Tanpa `LLM_PROVIDER=openai`, assistant tidak memanggil LLM apa pun — jawaban disusun deterministik dari konteks retrieval dan hasil tool. Ini disengaja untuk demo offline dan test.
3. **Vector store masih in-memory.** Indeks dibangun ulang setiap kali assistant start. Memadai untuk knowledge base kecil–sedang, belum untuk skala besar atau pembaruan inkremental.
4. **Baseline klasik masih mengalahkan MLP** pada dataset turnover saat ini (ROC AUC 0.72 vs 0.69). Modul `deep_learning` hadir sebagai pembanding yang jujur, bukan klaim kemenangan.
5. **Skor prediksi adalah sinyal awal**, bukan dasar keputusan otomatis (PHK, sanksi, atau promosi). Hal ini juga ditegaskan pada jawaban assistant.

## Lisensi

Private / Proprietary.

Hak penggunaan, distribusi, modifikasi, dan akses terhadap source code mengikuti kebijakan pemilik proyek.

© 2026 All Rights Reserved.